Sistemas Distribuidos

Ayudantía 6

Profesor Nicolas Hidalgo

Ayudante cátedra: Benjamín Aceituno, Vicente Díaz
Ayudante corrector: Isidora Gonzalez, Natalia Ortega

Contactos

- **Benjamín Aceituno**
  benjamin.aceituno@mail.udp.cl
- **Vicente Díaz**
  Vicente.diaz5@mail.udp.cl
- **Isidora González**
  isidora.gonzalez4@mail.udp.cl
- **Natalia Ortega**
  natalia.ortega1@mail.udp.cl



¿Por qué sincronizar?

Acceso a Recursos Compartidos

Múltiples procesos compiten por un mismo recurso (ej. impresora, base de datos). Sin orden, se producen condiciones de carrera e inconsistencias.

Orden de Eventos

Para mantener réplicas consistentes las escrituras deben aplicarse en el mismo orden en todas las copias. Sin sincronización se pierden datos.

Tiempo: magnitud escalar que mide el intervalo entre dos sucesos en un sistema bajo observación.

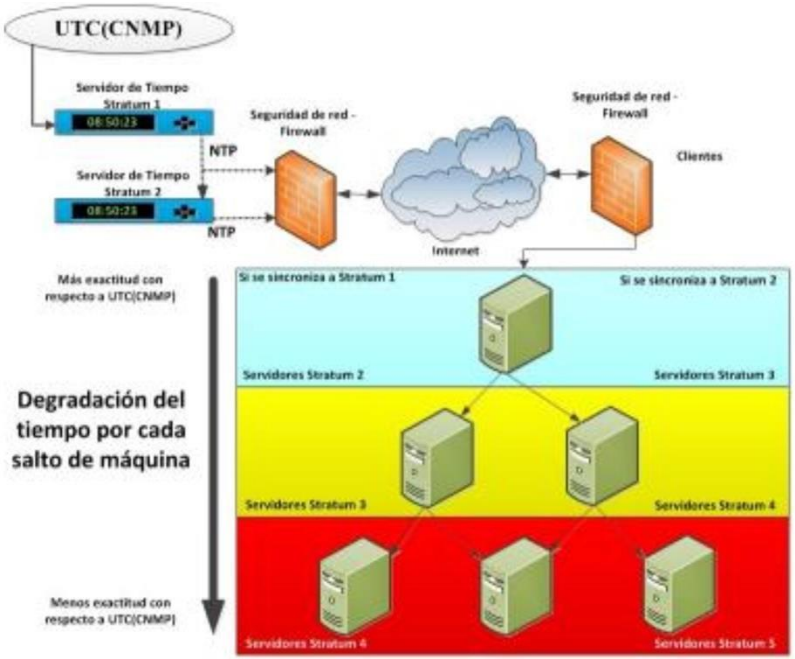


Figura 1. Esquema de comunicación del servicio de sincronización al UTC(CNMP).

Relojes Físicos

¿Cómo funcionan?

- Cristal de cuarzo + 2 registros (contador y almacenador)
- Cada oscilación decrementa el contador → genera una interrupción (tic)
- Al iniciar el computador se traduce la fecha a número de tics
- No pueden todos los cristales oscilar igual → variaciones físicas

Problemas

Clock Drift

Relojes se desajustan gradualmente

Clock Skew

Diferencia acumulada entre máquinas

Deriva

Cada cristal avanza a diferente velocidad

UTC (Coordinated Universal Time): estándar mundial derivado de relojes atómicos TAI con ajustes de leap seconds.

Algoritmo de Cristian

1

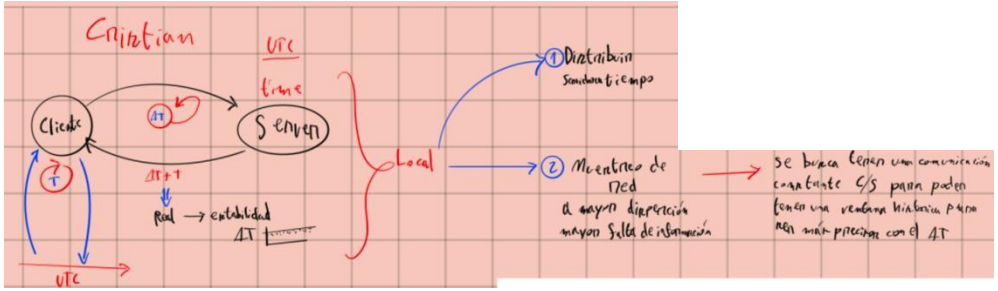
Request: El cliente envía una solicitud de tiempo al servidor UTC

2

Response: El servidor responde con su hora actual T

3

Adjust: Cliente estima: T_real ≈ T + RTT/2() (se asume latencia simétrica) ($T_{real} = T_{servidor} + (\frac{t3-t0}{2})$)



Limitaciones

- Solo redes pequeñas (LAN) — asume latencia simétrica
- Punto único de fallo

Algoritmo de Berkeley

No necesita UTC externo — sincronización interna entre procesos de la red

1. Consulta

Maestro/coordinador consulta la hora a todos los nodos de la red

2. Promedio

Calcula el promedio de tiempos (ignorando relojes muy desviados)

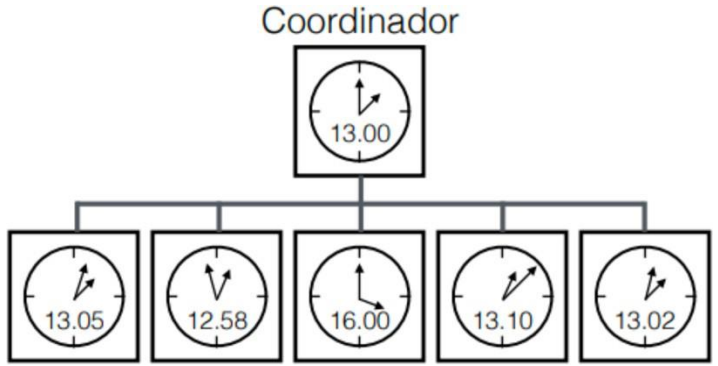
3. Offsets

Envía a cada nodo el offset a ajustar, NO la hora absoluta

4. Ajuste

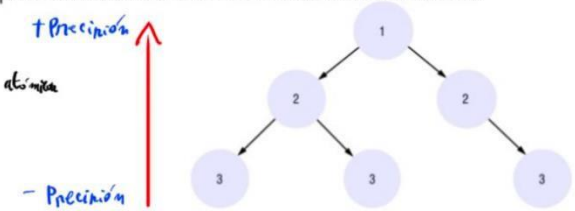
Cada nodo ajusta su reloj con el offset recibido

	Cristian	Berkeley
Necesita UTC	Sí	No
Rol servidor	Pasivo	Activo
Escala	LAN	Red interna

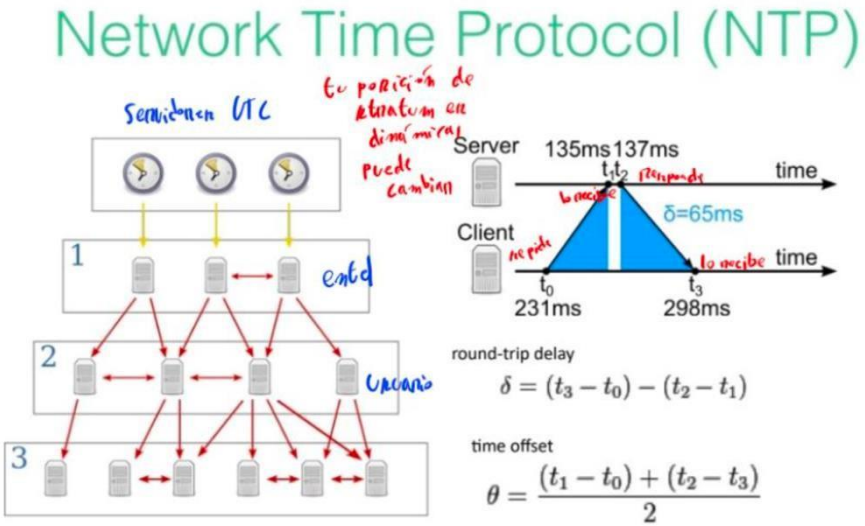


Network Time Protocol (NTP)

- Niveles de la jerarquía son conocidos como *strata o stratum*



Arrows denote synchronization control, numbers denote strata.



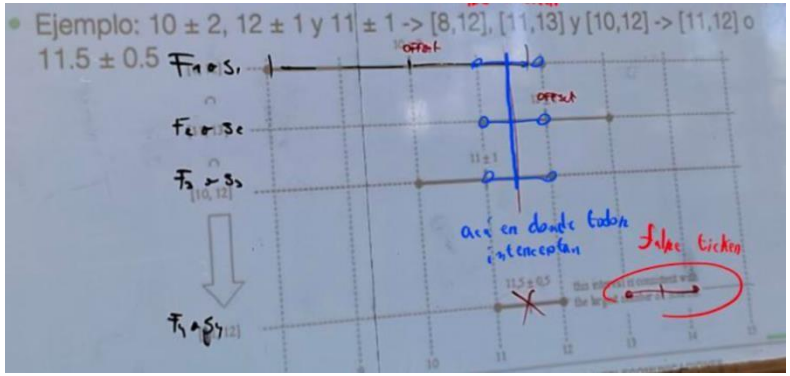
Modos NTP: Client/Server: chequea servidores para sincronizar | Symmetric: backup mutuo entre stratos bajos (polling) | Broadcast/Multicast: poca precisión, un servidor notifica a todos

Network Time Protocol (NTP)

¿Cómo funcionan?

- Paso 1: Muestreo (Sampling)
  Obtención de datos.
- Paso 2: Filtrado (Filtering)
  Elegir la mejor muestra por servidor (Retraso mínimo)
- Paso 3: Selección (Selection)
  Elegir el major servidor (Strata y FalseTicker):

  - Se calculan los tiempos de propagación aplicando el concepto de dispersion (suma de las dispersions servidor-root).
- Ajustes de reloj: se aplica una fase para la modificación de las frecuencias de los relojes en base a las derivadas observadas.



- NTP utiliza el “Algoritmo de Marzullo”
- Este algoritmo se utiliza para seleccionar fuentes de tiempo, relojes para sincronizar tiempo entre varios relojes, basado en un origen o fuente único.

3 modos de sincronización:

- Client/server: servidor chequea uno o más servidores para sincronizar.
- Symmetric: para stratos bajos de la arquitectura. Se utiliza un approach p2p entre ellos para sincronizar. Utiliza mecanismo polling.
- Broadcast/multicast: se usa cuando se requiere poca precisión. Se aplica a nivel de un servidor que aplica un broadcast. Uno sincroniza y notifica a todo el resto

Relojes Lógicos de Lamport

Objetivo: ordenar eventos — NO medir tiempo real. Define relación happened-before (→)

R1 **Orden local**

Si a y b son eventos del mismo proceso y a ocurre antes que b → a → b

R2 **Envío/Recepción**

El envío de un mensaje siempre ocurre antes que su recepción → a → b

R3 **Transitividad**

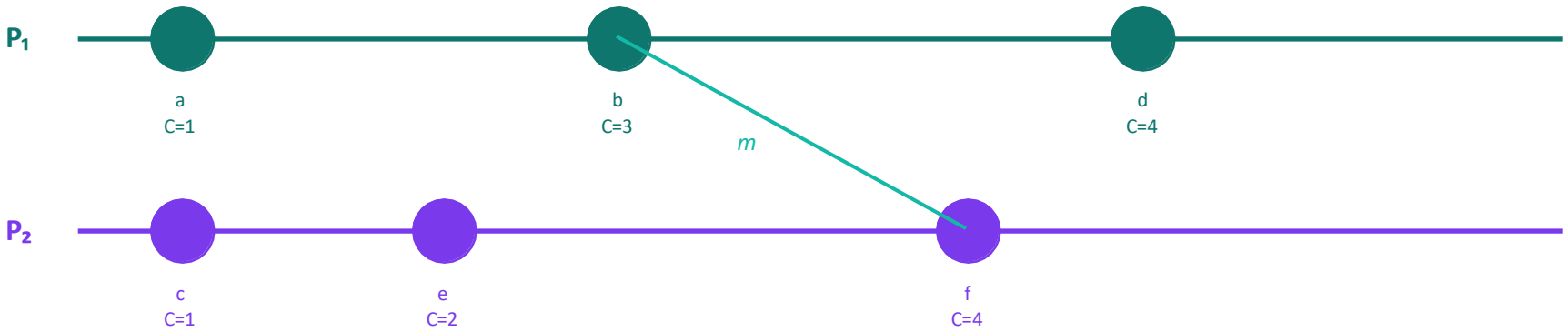
Si a → b y b → c, entonces a → c

Algoritmo

- Cada proceso mantiene un contador C (inicialmente 0)
- Evento local: C ← C + 1
- Al enviar mensaje: se adjunta el timestamp C
- Al recibir mensaje con timestamp Cm: C ← max(C_local, Cm) + 1

Limitación de Lamport

Si a → b, entonces C(a) < C(b)
Pero si C(a) < C(b), NO implica necesariamente a → b



Problema: c y a tienen C=1 pero no tienen relación causal. Lamport no puede distinguir eventos **concurrentes** de eventos que sí tienen causalidad → necesitamos Relojes Vectoriales.

Relojes Vectoriales

Cada proceso i mantiene un vector V de tamaño N (uno por proceso en el sistema)

Reglas

- Evento local en proceso i → V[i] ← V[i] + 1
- Envío de mensaje → se adjunta el vector V completo
- Recepción de mensaje con vector Vm →
- V[j] ← max(V[j], Vm[j]) para todo j
- luego V[i] ← V[i] + 1

Comparación de vectores

V(a) = V(b) → Mismo instante (raro)

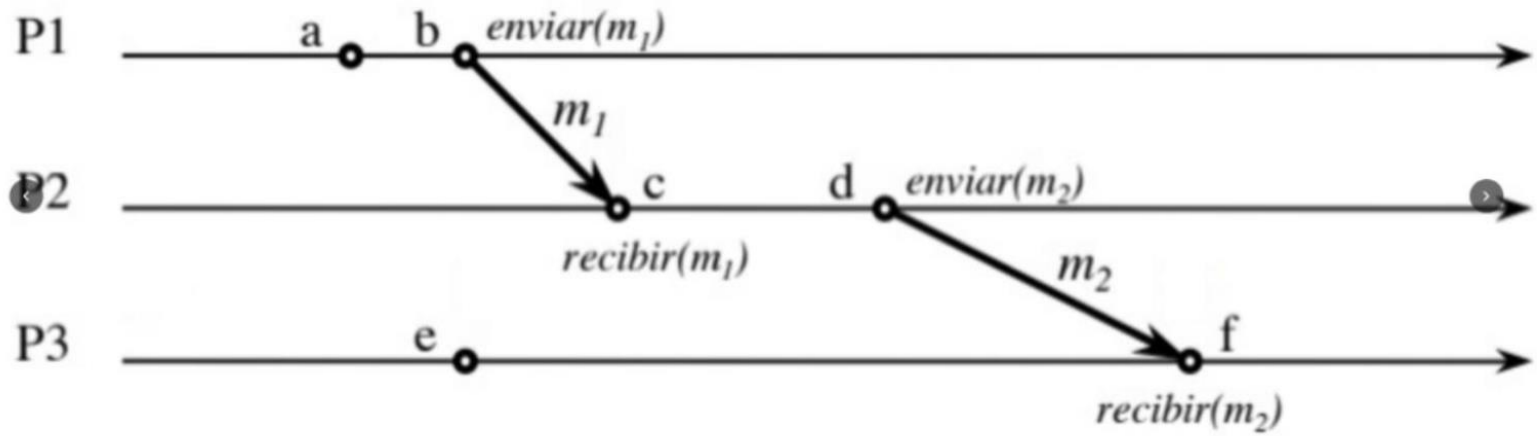
V(a) < V(b) → a causó b (a → b)

V(a) > V(b) → b causó a (b → a)

Ninguno ≤ otro → a ‖ b (concurrentes)

Reloj Lógico Lamport: C(a) < C(b) no implica causalidad
Reloj Vectorial: V(a) ≤ V(b) ⟺ a causó b (detecta concurrencia exactamente)

Ejemplo: Relojes Vectoriales



Multicast Totalmente Ordenado

Problema: en replicación, ¿cómo asegurar que todas las réplicas apliquen las actualizaciones en el mismo orden?

Solución: Totally Ordered Multicast

1

Envío con timestamp

Proceso envía mensaje a todos con su timestamp de Lamport

2

Cola local ordenada

Al recibir, el proceso inserta el mensaje en su cola local ordenado por timestamp

3

ACK broadcast

El receptor envía un ACK a todos los demás procesos

4

Entrega garantizada

Se entrega el mensaje solo cuando: (a) tiene el menor timestamp en la cola Y (b) se recibieron ACKs de todos

Garantía: todos los procesos entregan los mensajes en el mismo orden global — incluso sin relación causal

¡Muchas gracias!

Ayudantía · Sincronización en Sistemas Distribuidos